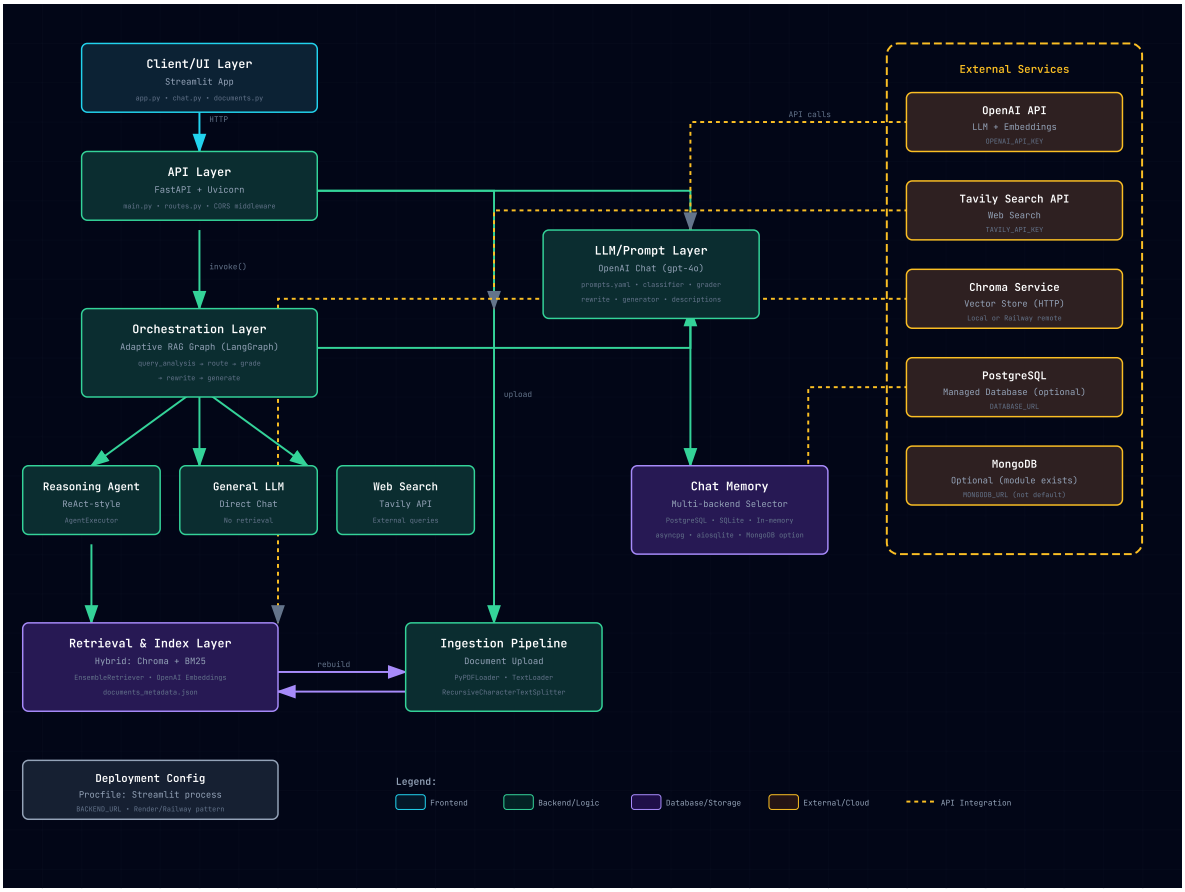
Adaptive RAG System Architecture

LangGraph + FastAPI + Streamlit | Multi-layer Orchestration with Hybrid Retrieval



Request Flow

- User query via Streamlit UI
- FastAPI receives HTTP request
- LangGraph graph analyzes intent
- Routes to retriever/LLM/search
- Grades results, rewrites if needed
- Generates final response
- Returns to client with citations

Key Technologies

- **Frontend:** Streamlit, requests
- **Backend:** FastAPI, Pydantic
- **Orchestration:** LangGraph, LangChain
- **Agent:** ReAct-style tool calling
- **Retrieval:** Chroma + BM25 hybrid
- **LLM:** OpenAI gpt-4o
- **Memory:** Postgres/SQLite/in-memory

Data Persistence

- **Vectors:** Chroma HTTP server

Environment Config

- **OPENAI_API_KEY:** LLM + embeddings

- **BM25:** In-memory index
- **Metadata:** documents_metadata.json
- **Chat history:** Auto-selects backend
- **Priority:** Postgres → SQLite → memory
- **Documents:** PDF/TXT chunked on upload
- **Embeddings:** OpenAI text-embedding

- **TAVILY_API_KEY:** Web search
- **CHROMA_HOST/PORT:** Vector store
- **DATABASE_URL:** PostgreSQL (opt)
- **MONGODB_URL:** MongoDB (opt)
- **BACKEND_URL:** API endpoint
- **DISABLE_AUTH:** Local mode flag

Adaptive RAG Architecture • Multi-layer LangGraph Orchestration • Hybrid Retrieval with Chroma + BM25